

LAB # 3 : Functions, Loops, and Basic Computation in R

Introduction

In this lab, we write and use user-defined functions in *R*. We apply loops (for, while) and conditionals. We also use *R* for basic probability, calculus, and simulation. You may find it convenient to print the pdf version of this lab rather than the web page itself.

Functions and Basic Computation

- We write a function which returns sum of two numbers.

```
add_num <- function(a,b)
{
  sum_result <- a+b
  return(sum_result)
}
# calling add_num function and printing the output
> add_num(2,pi)
[1] 5.141593
```

```
avg <- function(x){
  s <- sum(x)
  n <- length(x)
  s/n
}
# output
x=c(1,2,3)
avg(x)
[1] 2
```

- Define the function as an expression:

```
f <- expression(x^2 + 3*x + 5)
# Compute the derivative symbolically
f_prime <- D(f, "x")
print(f_prime)
# Evaluate the derivative at a specific point
x_val <- 2
eval(f_prime, list(x = x_val))
# OUTPUT
2*x+3
[1] 7
```

- We can also compute derivative by installing package *namley* "*Deriv*".

```
install.packages("Deriv")
library(Deriv)
# Define the function
f <- function(x) { x^2 + 3*x + 5 }
# Compute the derivative
f_prime <- Deriv(f, "x")
# Evaluate at a specific point
x_val <- 2
f_prime(x_val)
# Output
[1] 7
```

- We can find a root of a nonlinear equation using *uniroot*.

```
# Define the function
f <- function(x) { x - cos(x) }
# Use uniroot to find the root
root <- uniroot(f, interval = c(0, 3))
root$root
[1] 0.7390804
```

- We can compute integration using *integrate*.

```
f <- function(x) { x - cos(x) }
integral_value <- integrate(f, 0, 3)
print(integral_value$value)
[1] 4.35888
```

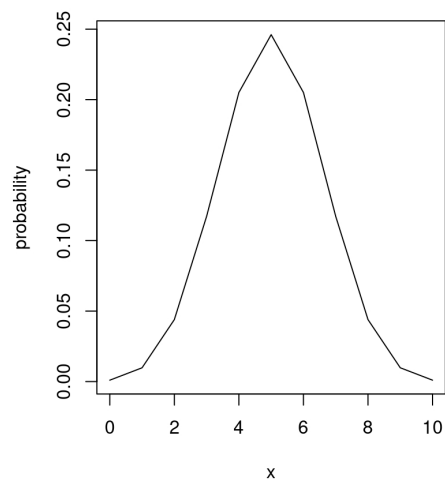
- We can plot the following function in *R*:

$$f(x) = \binom{n}{x} p^x (1-p)^{n-x}, \quad x = 0, 1, 2, \dots, n, \quad 0 < p < 1.$$

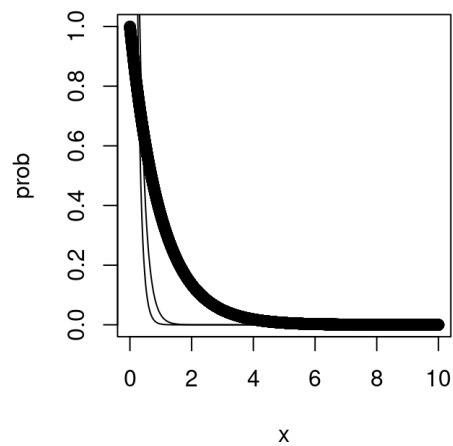
```
y <- function(n,p,x)
{y <- choose(n,x)*p^x*(1-p)^(n-x)
y}
x <- seq(0,10)
plot(x,y(10,0.5,x),"h") # l for line, p for point, o for both
```

- We can write $f(x) = \lambda e^{-\lambda x}$, $x > 0$, where λ is a positive parameter, in *R* and then plot it for different values of λ using *lines* command.

```
fun <- function(x,lam) {
fun=lam*exp(-lam*x)
}
x=seq(0,10,0.001)
#plot(x,fun(x,1))
plot(x,fun(x,1))
```



```
lines(x,fun(x,5))
lines(x,fun(x,8))
```



Loops and Conditional Statements

- If-Else Statement

```
a <- 5
b <- 7
if (b > a) {
  print("b is greater than a")
} else if (a == b) {
```

```

print ("a and b are equal")
} else {
print("a is greater than b")
}
# Output
[1] "b is greater than a"

```

- While Loop

```

i <- 1
while (i < 6) {
print(i)
i <- i + 1
}
# output
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5

```

- For Loop

```

for (x in 1:5){
print(x)
}
# output
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5

```

- Logical Operators in R

<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	exactly equal to
!=	not equal to
x & y	x and y
	or

Example: Write a *R* program to calculate the factorial of given number.

```

num <- 5
fact <- 1
if(num<0){

```

```

print("fact does not exist")
}else if (num==0){
print("fact of 0 is 1")
} else {
for (i in 1:num){
fact=fact*i
}
print(paste("the factorial of",num,"is",fact))
}
# output
[1] "the factorial of 5 is 120"

```

Sample Command: The command *sample()* is used to generate the random elements from the given data with or without replacement.

```
sample(data, size, replace = FALSE, prob = NULL)
```

Let us suppose a box contains 10 white, 20 blue, and 35 black balls. You randomly pick 10 balls from the box. Write the sample space.

```

balls <- c(rep("white", 10),rep("blue", 20), rep("black",35))
sample(balls,size=10, replace=FALSE)
# Output
[1] "blue" "black" "white" "blue" "black" "black" "black"
[8] "white" "blue" "black"

```

Problems:

1. A company has three factories (*A*, *B* and *C*) that produce the same chip, each producing 15%, 35% and 50% of the total production. The probability of a defective chip at 1, 2, 3 is 0.01, 0.05, 0.02, respectively. Suppose someone shows us a defective chip. What is the probability that this chip comes from factory *A*? Write a FUNCTION in *R* which calculate the probability.
2. The normal probability density function is given by

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad -\infty < x < \infty.$$

- (a) Write an R function that computes the normal density for given values of x , mean μ , and standard deviation σ .
 - (b) Plot the density on the interval $[-4, 4]$ for $\mu = 0$ and $\sigma = 1$.
 - (c) On the same graph, add the density for $\mu = 0$ and $\sigma = 2$ using the `lines()` command.
3. Define the function

$$f(x) = \begin{cases} x^2, & x < 0, \\ x, & x \geq 0. \end{cases}$$

- (a) Write an R function that evaluates $f(x)$ using an `if` statement.
- (b) Use a `for` loop to evaluate the function at the points $x = -5, -4, \dots, 5$.
- (c) Plot the function over the interval $[-5, 5]$.